

CONTRACTIQ

Contract Obligation Tracking & Compliance Management Platform

Sprint 3 – Database Design

By Anshika Kumari

PostgreSQL • SQLAlchemy • FastAPI • Angular

1. Purpose of the Database

The ContractIQ database is designed around the contract lifecycle implemented by the application: authenticated users manage contracts, contracts contain trackable obligations and renewal information, compliance is calculated from obligation status, notifications communicate important events, reports provide analytics and exports, and audit logs provide traceability.

The database uses PostgreSQL with SQLAlchemy ORM. Integer identifiers and foreign-key relationships are used for consistent integration with the FastAPI backend. Date and timestamp fields support lifecycle tracking, while status fields support dashboard analytics and operational workflows.

The design is intentionally modular so that each business responsibility is represented separately while relationships preserve the connection between a contract and its related operational data.

2. Database Tables & Application Modules

#	Table / Layer	Project Purpose
1	users	Authentication, user identity, role and account status
2	contracts	Central contract master records and lifecycle status
3	contract_versions	Contract document/version history
4	obligations	Contractual duties, deadlines, assignment and status
5	renewals	Renewal dates, expiry information and renewal status
6	notifications	User-specific alerts and notification state
7	audit_logs	Traceability of important system actions
8	activities	Operational timeline for contract/obligation activity
9	reports / analytics layer	Report generation and analytics; current implementation exposes report APIs and PDF/Excel exports

Implementation note: ContractIQ currently generates analytics and PDF/Excel exports through the reporting service and API layer. A dedicated reports metadata table can be retained as an extensibility option, but it is not required for the current export workflow.

3. Core Database Schema

3.1 users

Stores platform users used for authentication, authorization, contract ownership and assignment.

Column	Data Type	Key / Constraint	Project Purpose
id	INTEGER	PK	Unique user ID
full_name	VARCHAR(100)	NOT NULL	User full name
email	VARCHAR(255)	UNIQUE, NOT NULL	Login/contact email
password	VARCHAR(255)	NOT NULL	Stored password value used by the current backend authentication model
role	VARCHAR(50)	NOT NULL	User role
is_active	BOOLEAN	NOT NULL	Account availability
created_at	TIMESTAMP	NOT NULL	Account creation timestamp
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

3.2 contracts

Central business entity. Most contract lifecycle information is linked to this table.

Column	Data Type	Key / Constraint	Project Purpose
id	INTEGER	PK	Unique contract ID
contract_number	VARCHAR(100)	UNIQUE, NOT NULL	Unique contract reference
title	VARCHAR(255)	NOT NULL	Contract title

category	VARCHAR(255)	NOT NULL	Contract category
description	TEXT	NULL	Contract description
party_name	VARCHAR(255)	NOT NULL	Other contracting party
start_date	DATE	NOT NULL	Contract start date
end_date	DATE	NOT NULL	Contract expiry date
status	VARCHAR(50)	NOT NULL	Draft, Under Review, Approved, Active, Expired, Terminated
owner_id	INTEGER	FK → users.id	Primary contract owner
assigned_to	INTEGER	FK → users.id, NULL	Assigned user
reviewed_at	TIMESTAMP	NULL	Review timestamp
approved_at	TIMESTAMP	NULL	Approval timestamp
created_at	TIMESTAMP	NOT NULL	Creation timestamp
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

3.3 obligations

Tracks contractual responsibilities and provides the main input for compliance calculation.

Column	Data Type	Key / Constraint	Project Purpose
id	INTEGER	PK	Unique obligation ID
contract_id	INTEGER	FK → contracts.id	Associated contract
title	VARCHAR(255)	NOT NULL	Obligation title
description	TEXT	NULL	Detailed obligation description
obligation_type	VARCHAR(100)	NOT NULL	Payment, delivery, reporting, renewal, SLA, legal compliance, etc.
due_date	DATE	NOT NULL	Obligation deadline
assigned_to	INTEGER	FK → users.id	Responsible user
status	VARCHAR(50)	NOT NULL	Pending, In Progress, Completed, Delayed, Overdue
completion_date	DATE	NULL	Completion date
created_at	TIMESTAMP	NOT NULL	Creation timestamp
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

3.4 renewals

Tracks renewal events and connects contract expiry with renewal management.

Column	Data Type	Key / Constraint	Project Purpose
id	INTEGER	PK	Unique renewal ID
contract_id	INTEGER	FK → contracts.id	Contract being renewed
renewal_date	DATE	NOT NULL	Renewal date
previous_expiry_date	DATE	NOT NULL	Previous expiry date
new_expiry_date	DATE	NOT NULL	New expiry date
status	VARCHAR(50)	NOT NULL	Renewal lifecycle status
assigned_to	INTEGER	FK → users.id	Responsible user
notes	TEXT	NULL	Renewal notes
created_at	TIMESTAMP	NOT NULL	Creation timestamp
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

3.5 notifications

Stores user-scoped alerts related to contracts, obligations and system events.

Column	Data Type	Key / Constraint	Project Purpose
id	INTEGER	PK	Unique notification ID
user_id	INTEGER	FK → users.id	Notification recipient
contract_id	INTEGER	FK → contracts.id, NULL	Related contract
obligation_id	INTEGER	FK → obligations.id, NULL	Related obligation
notification_type	VARCHAR(100)	NOT NULL	Notification category
title	VARCHAR(255)	NOT NULL	Notification title

message	TEXT	NOT NULL	Notification message
status	VARCHAR(50)	NOT NULL	Notification status
scheduled_at	TIMESTAMP	NULL	Scheduled time
sent_at	TIMESTAMP	NULL	Sent time
read_at	TIMESTAMP	NULL	Read time
created_at	TIMESTAMP	NOT NULL	Creation timestamp
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

3.6 audit_logs

Captures important actions for accountability and traceability.

Column	Data Type	Key / Constraint	Project Purpose
id	BIGINT	PK	Unique audit event
user_id	INTEGER	FK → users.id, NULL	Actor; may be null for system actions
action	VARCHAR(100)	NOT NULL	Action such as CREATE or UPDATE
entity_type	VARCHAR(100)	NOT NULL	Affected entity
entity_id	INTEGER	NULL	Affected record ID
old_values	JSONB	NULL	Previous state
new_values	JSONB	NULL	New state
ip_address	VARCHAR(45)	NULL	Origin IP when captured
created_at	TIMESTAMP	NOT NULL	Event timestamp

4. Supporting Tables & Extensibility

4.1 contract_versions

The project design reserves contract_versions for maintaining historical document versions. A contract can have multiple versions, each connected back to contracts.id and the user who created the version.

- contract_id → contracts.id
- version_number identifies the sequence of a contract version
- file_path stores the document reference
- change_summary describes changes made in the version
- created_by → users.id identifies the creator

4.2 activities

The activity model supports a chronological operational timeline for contracts and obligations. It can capture actions such as comments, assignments, uploads and status changes.

- user_id → users.id
- contract_id → contracts.id (optional)
- obligation_id → obligations.id (optional)
- activity_type and description provide the human-readable event information
- created_at provides chronological ordering

4.3 Reports / Analytics

In the current implementation, reports are primarily produced through the FastAPI reporting service. The application exposes contract, obligation, renewal, compliance and risk analytics and supports PDF and Excel exports. This keeps report generation reusable without requiring a stored report record for every download.

5. Primary Keys & Foreign Keys

Table	Primary Key	Foreign Keys
users	id	—
contracts	id	owner_id → users.id; assigned_to → users.id
contract_versions	id	contract_id → contracts.id; created_by → users.id
obligations	id	contract_id → contracts.id; assigned_to → users.id
renewals	id	contract_id → contracts.id; assigned_to → users.id
notifications	id	user_id → users.id; contract_id → contracts.id; obligation_id → obligations.id
audit_logs	id	user_id → users.id
activities	id	user_id → users.id; contract_id → contracts.id; obligation_id → obligations.id

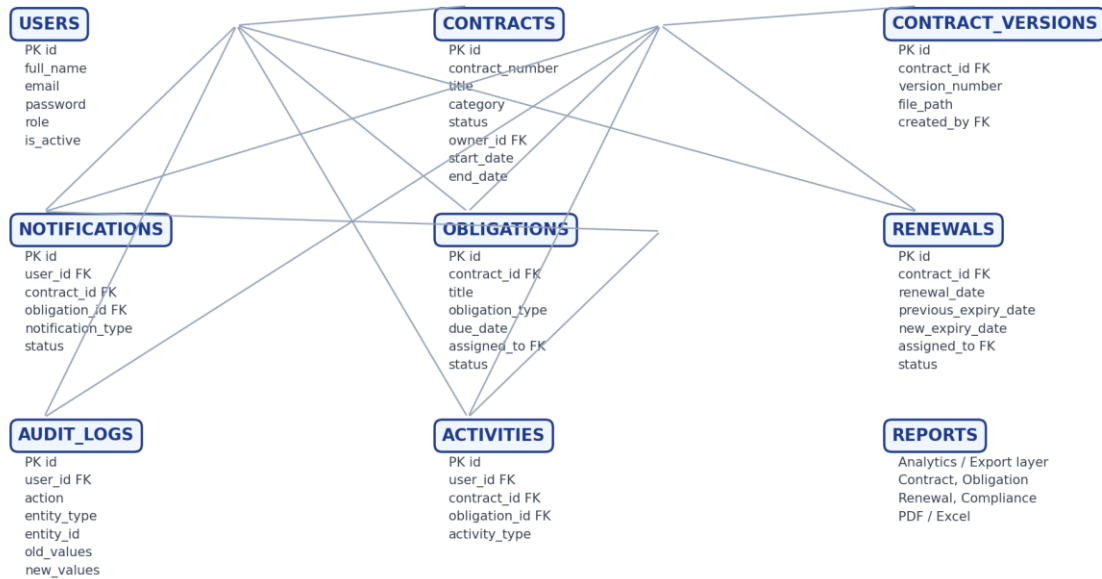
6. Relationships Between Tables

Parent	Child	Relationship	Explanation
users	contracts	1 : N	One user can own or be associated with multiple contracts.
contracts	contract_versions	1 : N	One contract can have multiple document versions.
contracts	obligations	1 : N	One contract can contain multiple obligations.
users	obligations	1 : N	One user can be assigned multiple obligations.
contracts	renewals	1 : N	One contract can have multiple renewal records.
users	renewals	1 : N	One user can be responsible for multiple renewals.
users	notifications	1 : N	One user can receive multiple notifications.
contracts	notifications	1 : N	A contract can have multiple related notifications.
obligations	notifications	1 : N	An obligation can have multiple related notifications.
users	audit_logs	1 : N	A user can generate multiple audit events.
contracts	activities	1 : N	A contract can have multiple activity events.
obligations	activities	1 : N	An obligation can have multiple activity events.

7. Entity Relationship Diagram

The diagram below reflects the ContractIQ domain model: users provide ownership, assignment and audit references; contracts act as the central business entity; obligations and renewals extend the contract lifecycle; notifications connect operational events to users; and audit/activity records provide traceability.

ContractIQ - Database Relationship Overview



8. Functional Interaction of the Database

ContractIQ follows a contract-centered data flow. A user authenticates and accesses protected application modules. Contracts are created and maintained as the central records. Obligations are attached to contracts and assigned to users. Obligation completion and overdue status feed compliance analysis. Renewals track future contract lifecycle events. Notifications communicate relevant information to users. Dashboard and reporting APIs aggregate these records for analytics. Audit logs provide a traceable record of important actions.

8.1 Application Data Flow

- User → authentication and protected application access
- Contract → master record containing lifecycle and ownership information
- Obligation → operational task linked to a contract and responsible user
- Renewal → future lifecycle event linked to the contract
- Compliance → derived analysis from contract obligations
- Notification → user-facing communication linked to relevant records
- Dashboard / Reports → aggregated analytics and export layer
- Audit History → traceability of important application actions

9. Constraints & Design Decisions

- contract_number must be unique so that each contract has a stable business reference.
- Contract start and end dates represent the lifecycle period and should be logically consistent.
- Foreign-key columns should be indexed where frequently used for joins and lookups.
- Obligation and renewal status values are restricted through application validation.
- Audit records should be treated as append-only from the application layer.
- Passwords must not be stored as plaintext; authentication should use secure password handling.
- Notification records are scoped to users so that one user does not access another user's notifications.

- Sensitive configuration such as database credentials and JWT secrets belongs in environment variables, not source control.

10. Database-to-Backend Integration

The database is accessed through SQLAlchemy models and a FastAPI service/router architecture. Pydantic schemas validate request and response data, while the reporting and compliance services transform database records into application-level analytics.

Domain	Backend Layer	Purpose
Contracts	Contract model / router	Create, read, update and lifecycle tracking
Obligations	Obligation model / router	Create, update, status and completion workflows
Renewals	Renewal model / router	Create, update, status and renewal workflows
Compliance	Compliance service / router	Contract compliance and risk calculations
Notifications	Notification model / router	User notification retrieval and read actions
Reports	Report service / router	Analytics, PDF and Excel generation
Audit	AuditLog model / audit router	Authenticated audit history retrieval

11. Alignment with the Implemented ContractIQ Application

This database design is aligned with the current full-stack ContractIQ implementation. The Angular frontend consumes FastAPI APIs through dedicated services, while the FastAPI backend uses SQLAlchemy and PostgreSQL for persistence. Dashboard and reporting values are derived from backend data rather than hardcoded frontend values.

- PostgreSQL is the primary relational database.
- FastAPI provides the REST API layer.
- SQLAlchemy provides ORM-based database access.
- JWT authentication protects application APIs and frontend routes.
- Contract, obligation, renewal, compliance and notification modules are integrated across the application.
- Reporting supports analytics plus PDF and Excel export.
- Audit History provides a frontend view of audit log records.
- The Department Performance requirement is not represented as a database feature because the current schema does not contain department / department_id data.

12. Conclusion

The ContractIQ database design provides a normalized, relationship-driven structure for contract lifecycle management. Contracts remain the central business entity, while obligations, renewals, notifications, compliance analysis, reporting and audit history extend the platform without mixing unrelated responsibilities. The design supports the current FastAPI + Angular implementation and leaves clear extension points for document versioning, activity timelines, department analytics and richer report metadata.